ECE 408

第 12 讲

Computation in Large Language Models

目录

1 自回归生成与模型输入 **1**
1.1 从条件分布到逐步生成 . . . 1
1.2 分词、嵌入与位置信息 . . . 2
1.3 解码器结构与张量约定 . . . 3

2 因果多头注意力与层内计算 **4**
2.1 查询、键和值的投影 . . . 4
2.2 缩放、因果掩码与加权聚合 . . . 4
2.3 多个注意力头与特征融合 . . . 5
2.4 残差连接、层归一化与前馈网络 . . . 6
2.5 三个词元的注意力算例 . . . 7

3 注意力的 CUDA 实现 **8**
3.1 矩阵乘法流水线与存储布局 . . . 8
3.2 数值稳定的因果归一化 . . . 9
3.3 一行一个线程块的计算组织 . . . 10
3.4 启动条件与实现检查 . . . 13

4 两阶段推理与键值缓存 **14**
4.1 预填充与增量解码的时序 . . . 14
4.2 历史状态为什么可以复用 . . . 15
4.3 一个新增位置在各层中的计算 . . . 15
4.4 节省的计算与增加的状态容量 . . . 17
4.5 变长序列与键值头共享 . . . 18

5 计算规模与端到端性能 **19**
5.1 矩阵形状如何决定运算量 . . . 19
5.2 大规模投影的参数与容量算例 . . . 20
5.3 预填充与解码的资源需求 . . . 20
5.4 分块融合如何减少中间矩阵读写 . . . 22
5.5 从核函数性能到完整生成时间 . . . 23

1 自回归生成与模型输入

1.1 从条件分布到逐步生成

大语言模型（Large Language Model，LLM）把文本生成表示为一个序列概率问题。分词器（tokenizer）先把文本转换为词元（token）的整数编号序列。词元可以对应一个字、一个子词、一段标点或若干字节；词元边界由分词规则决定，通常与自然语言中的词边界并不完全一致。设词表大小为 V_{voc}，一个长度为 N 的已知序列写为 $t_0, t_1, \dots, t_{N-1}$，其中 $t_i \in \{0, \dots, V_{\text{voc}} - 1\}$。

自回归（autoregressive）模型根据已有前缀预测下一个词元。对于一个完整序列，其联合概率可以按概率的链式法则分解为

$$p(t_0, \dots, t_{T-1}) = \prod_{i=0}^{T-1} p(t_i \mid t_0, \dots, t_{i-1}). \tag{1}$$

当 $i = 0$ 时，条件为空，或由约定的起始标记提供。模型中的可训练参数共同确定这些条件分布。推理时固定参数，把系统输入、用户输入以及已经生成的词元共同作为上下文（context），反复计算下一词元的分布。

设完成最终归一化后，位置 i 的表示向量为 $h_i \in \mathbb{R}^D$，D 为模型宽度。词表投影把它变为未归一化分数（logits）$\ell_i = h_i W_U + b_U$，其中 $W_U \in \mathbb{R}^{D \times V_{\text{voc}}}$，$b_U \in \mathbb{R}^{V_{\text{voc}}}$ 为偏置向量。再经过软最大归一化（softmax）得到

$$p(t_{i+1} = v \mid t_0, \dots, t_i) = \frac{\exp(\ell_{i,v})}{\sum_{u=0}^{V_{\text{voc}}-1} \exp(\ell_{i,u})}. \tag{2}$$

这里位置 i 的输出对应位置 $i+1$ 的预测。这个错开一位的关系解释了因果注意力为什么允许位置 i 读取它自身的输入。

生成过程还需要一个解码规则。贪心解码取分数最大的词元；随机采样则按照归一化后的分布选择词元。若采用温度 $\tau > 0$，先把分数替换为 ℓ_i / τ，再计算分布。选出的词元追加到前缀后，进入下一次推理；遇到序列结束标记（end-of-sequence，EOS）、达到长度限制或满足其他既定停止条件时终止。词表分数的最大值位置在软最大归一化前后相同，因此只需贪心选择时可以直接对分数求最大值。

训练使用已知文本，可以同时构造许多位置的下一词元预测。推理时未来词元尚未确定，生成下一步的输入依赖当前步的选择。因此，同一已知前缀内部可以并行计算多个位置，连续的新词元生成仍存在步骤之间的依赖。

1.2 分词、嵌入与位置信息

分词器处理离散的符号边界与编号。字节对编码（Byte Pair Encoding，BPE）从较小的符号单元出发，根据语料中的共现统计反复合并高频相邻单元，形成词表和合并次序。推理时按照固定的规则把输入编码成编号。GPT-2 使用字节层面的 BPE；对同一段文字，具体编号必须与预训练模型使用的词表一致，不能通过临时给词语编号来替代。

词元嵌入（token embedding）把编号映射到连续向量。设嵌入矩阵为 $E \in \mathbb{R}^{V_{\text{voc}} \times D}$，编号 t_i 的嵌入就是第 t_i 行 $E_{t_i,:}$。这等价于用独热向量左乘 E，实现时直接查表即可。编号本身只标识表中的行，编号之差不表示语义距离。嵌入向量之间的方向或内积关系可以携带训练得到的相似性，但一个坐标不必对应一个可独立命名的语义属性。

词表和嵌入表承担不同的任务。分词规则及合并次序通常在分词器构建阶段确定，随后保持固定；浮点嵌入参数通过语言模型的训练目标学习。推理既不重新学习词表，也不更新嵌入权重。GPT-2 小型模型的嵌入宽度为 $D = 768$，所以一个整数编号被查表为一个含 768 个浮点分量的向量。

词元嵌入本身不区分同一词元出现在哪个位置。位置信息需要另外提供。采用可训练的绝对位置嵌入时，设位置表为 $E^{\text{pos}} \in \mathbb{R}^{N_{\max} \times D}$，其中 $N_{\max}$ 为模型支持的最大位置数。模型的初始输入为

$$X^{(0)}_{i,:} = E_{t_i,:} + E^{\text{pos}}_{i,:}, \qquad 0 \le i < N \le N_{\max}. \tag{3}$$

GPT-2 采用这种相加方式。嵌入矩阵 $X^{(0)} \in \mathbb{R}^{N \times D}$ 的行保留序列顺序，列表示特征。位置编码还可以采用其他形式；对这里的计算分析，关键条件是每个词元的位置在前缀扩展后保持一致。

查表与相加对每个位置读取并写入 D 个分量，工作量为 $\Theta(ND)$。后续线性层常需 $\Theta(ND^2)$ 的算术工作，因此嵌入通常不是主体计算。不过，词表投影需要把宽度 D 的表示映射到整个词表，涉及矩阵乘法，其开销应与输入查表区分。

1.3 解码器结构与张量约定

变换器（Transformer）架构利用注意力在不同位置之间传递信息。编码器把完整输入序列转换为上下文相关表示；在编码器与解码器组合的结构中，解码器还可以通过交叉注意力（cross-attention）读取编码器输出，并根据已生成前缀产生目标序列。仅含解码器的变换器（decoder-only Transformer）把输入提示和生成结果置于同一个序列中，通过因果自注意力（causal self-attention）建立依赖。GPT 系列的生成计算属于后一种结构。

设网络包含 L 个变换器层。每层接收 $N \times D$ 的表示，输出形状保持为 $N \times D$，从而能够逐层堆叠。层内的多头注意力负责不同位置之间的信息混合，逐位置前馈网络负责每个位置内部的特征变换；残差连接和层归一化使这些变换能够稳定地组合。最终归一化和词表投影把最后一层表示转成下一词元分布。

表 1 张量与计数符号。矩阵公式默认省略批量维，向量按行参与乘法。

符号	含义	形状或关系
B, N, L	批量大小、上下文长度、层数	均为正整数
D, h, d	模型宽度、注意力头数、单头维度	$D = hd$
X	某一层或子层的输入	$N \times D$
$W_Q^{(a)}, W_K^{(a)}, W_V^{(a)}$	第 a 个头的投影矩阵	每个为 $D \times d$
$Q^{(a)}, K^{(a)}, V^{(a)}$	单头的查询、键和值	每个为 $N \times d$
$S^{(a)}, P^{(a)}$	单头分数矩阵、注意力权重	每个为 $N \times N$
$O^{(a)}$	单头输出	$N \times d$
D_{ff}	前馈网络的中间宽度	常取为 D 的若干倍
V_{voc}	词表大小	与值矩阵 $V^{(a)}$ 区分

例如，$N = 4096$、$D = 4096$、$h = 32$ 时，$d = 128$。输入仍然具有 4096 个特征；每个注意力头把这 4096 个特征投影到 128 维空间。所有头的输出拼接后恢复到 $32 \times 128 = 4096$ 维。上下文长度和模型宽度恰好相等只是这个例子的取值，二者在定义和计算作用上独立。

2 因果多头注意力与层内计算

2.1 查询、键和值的投影

自注意力（self-attention）根据当前输入决定不同位置之间的混合权重。对一个注意力头，设输入为 $X \in \mathbb{R}^{N\times D}$。通过三个独立的可训练投影得到查询（query）、键（key）和值（value）：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad W_Q, W_K, W_V \in \mathbb{R}^{D\times d}. \tag{4}$$

这里暂时省略头编号，并省略偏置项，以突出矩阵乘法；实际模型若含有投影偏置，仍需按行相加。每个输出矩阵均为 $N \times d$。写到元素层面，$Q_{i,r} = \sum_{c=0}^{D-1} X_{i,c}(W_Q)_{c,r}$，所以投影只对同一位置的特征做线性组合，还没有混合不同位置的信息。

查询 $q_i = Q_{i,:}$ 决定位置 i 如何评价其他位置；键 $k_j = K_{j,:}$ 提供位置 j 用于匹配的特征；值 $v_j = V_{j,:}$ 则提供匹配之后要组合的内容。三者由同一序列产生，所以称为自注意力。在交叉注意力中，查询和键值可以来自两个不同的表示序列。

查询与键的内积 $q_i k_j^{\mathrm{T}}$ 给出有方向的匹配分数。由于 W_Q 与 W_K 一般不同，$q_i k_j^{\mathrm{T}}$ 与 $q_j k_i^{\mathrm{T}}$ 不要求相等。因此，注意力矩阵的第 i 行描述“位置 i 如何读取其他位置”，不应理解成一个必然对称的相似度矩阵。值向量随后按这些权重被组合；最终加权的对象是 V 中的行。

卷积通常在固定局部范围内，使用训练后固定且跨位置共享的核系数来混合输入。自注意力的混合权重由当前输入重新计算，可以覆盖掩码允许的全部位置。两者都可以处理变化的序列长度，其主要差别在于连接范围以及混合系数如何确定。

2.2 缩放、因果掩码与加权聚合

先计算缩放分数 $S = QK^{\mathrm{T}}/\sqrt{d} \in \mathbb{R}^{N\times N}$。缩放因子由单头维度 d 决定。为理解其作用，暂设查询和键的各分量独立，均值为零、方差为一，并且不同维度的乘积互不相关。于是每项 $q_{i,r}k_{j,r}$ 的方差为一，包含 d 项的点积方差为 d，标准差为 $\sqrt{d}$。除以 $\sqrt{d}$ 后，分数尺度不再仅因头维度增加而增长。

这个估算说明了缩放的动机，不要求训练后的分量严格满足独立分布。若分数尺度过大，指数归一化容易把概率集中到少数位置，其他位置的梯度也会变小。$1/\sqrt{d}$ 作用于每个点积，与后续归一化中减去行最大值分别解决尺度和数值稳定性问题。

因果掩码（causal mask）限制位置 i 只能读取 $0, \ldots, i$。用加性掩码 M 表示这个约束，单

头的完整计算为

$$
\begin{aligned}
M_{ij} &= \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} & P &= \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d}} + M\right), \\
O &= PV, & O_{i,:} &= \sum_{j=0}^{i} P_{ij} v_j .
\end{aligned}
\tag{5}
$$

下标 row 表示独立归一化每一行。对于有效位置 $j \le i$，$P_{ij} \ge 0$，且 $\sum_{j=0}^{i} P_{ij} = 1$；对于 $j > i$，$\exp(-\infty) = 0$，因此权重为零。P 是包含对角线的下三角矩阵。

对角线保留是因为当前位置的词元已经是输入。例如，已知 t_0, t_1, t_2 时，位置 2 的输出用来预测 t_3，读取 t_2 不会读取到预测目标。若把对角线也屏蔽，第一行将没有任何有效位置，同时改变模型原本的条件分布。

从形状上看，第一项乘法 $(N \times d)(d \times N)$ 把特征维上的匹配变成位置之间的关系；第二项乘法 $(N \times N)(N \times d)$ 再把这些关系应用到值向量上。结果 $O \in \mathbb{R}^{N \times d}$ 的每一行是可见值向量的凸组合。多个头的拼接、输出投影和残差相加继续变换这个结果，所以完整层的输出不受这个单头凸组合性质直接约束。

2.3 多个注意力头与特征融合

多头注意力（multi-head attention）包含 h 个头，编号为 $a = 0, \ldots, h-1$。每个头分别配置查询、键和值的投影矩阵 $W_Q^{(a)}$、$W_K^{(a)}$ 和 $W_V^{(a)}$。各头从相同的输入构造不同的查询、键和值，在不同的投影空间中形成各自的权重分布。它们可以学习不同位置关系和特征关系；这些关系由训练形成，不需要给每个头指定一种分类任务。

各头在给定输入后可以独立计算。令第 a 个头的输出为 $O^{(a)} \in \mathbb{R}^{N \times d}$，沿特征方向拼接，再经过输出投影：

$$
Z = \mathrm{Concat}\left(O^{(0)}, \ldots, O^{(h-1)}\right) \in \mathbb{R}^{N \times hd}, \qquad \mathrm{MHA}(X) = ZW_O + b_O, \tag{6}
$$

其中 $W_O \in \mathbb{R}^{hd \times D}$、$b_O \in \mathbb{R}^{D}$，偏置按行广播。标准设置令 $hd = D$，所以输出形状回到 $N \times D$，能够与输入残差相加。拼接保留不同头的通道，W_O 再把它们线性混合。

在存储层面，可以把所有头的查询投影横向拼成 $\overline{W}_Q \in \mathbb{R}^{D \times D}$，一次计算 $X\overline{W}_Q \in \mathbb{R}^{N \times D}$，再将每行的 D 个分量分成 h 组、每组 d 个分量。键和值可以同样处理；进一步把三组权重拼为 $W_{QKV} \in \mathbb{R}^{D \times 3D}$，就能用一次矩阵乘法得到全部投影。逻辑上的多头独立与物理实现中的融合计算彼此兼容。

2.4 残差连接、层归一化与前馈网络

注意力只是一个完整层中的子层。残差连接（residual connection）把子层变换的结果与其输入逐元素相加，例如 $X + F(X)$。这样，原有表示可以直接传递，同时由 F 提供更新量；相加的两个矩阵必须具有相同形状。归一化与残差的先后顺序会改变实际函数，因此实现时必须固定顺序。

GPT-2 使用前置层归一化（pre-layer normalization）。若 $X^{(\ell)}$ 是第 ℓ 层输入，则完整层写为

$$\begin{aligned} U^{(\ell)} &= X^{(\ell)} + \mathrm{MHA}_{\ell}\big(\mathrm{LN}_{\ell,1}(X^{(\ell)})\big), \\ X^{(\ell+1)} &= U^{(\ell)} + \mathrm{FFN}_{\ell}\big(\mathrm{LN}_{\ell,2}(U^{(\ell)})\big). \end{aligned} \tag{7}$$

这里每个层归一化算子具有自己的一组参数。在式 (4) 中，X 表示送入注意力投影的矩阵；对应到式 (7)，它就是归一化后的 $\mathrm{LN}_{\ell,1}(X^{(\ell)})$。另一种常见的后置形式把子层写为 $\mathrm{LN}(X+F(X))$。两种放置方式的张量形状相同，但数值结果不同。

层归一化（layer normalization）独立处理每个位置的特征。对一行 $x_i \in \mathbb{R}^D$，定义

$$\begin{aligned} \mu_i &= \frac{1}{D}\sum_{c=0}^{D-1} x_{i,c}, \qquad \sigma_i^2 = \frac{1}{D}\sum_{c=0}^{D-1}(x_{i,c}-\mu_i)^2, \\ \mathrm{LN}(x_i)_c &= \gamma_c \frac{x_{i,c}-\mu_i}{\sqrt{\sigma_i^2+\varepsilon}} + \beta_c. \end{aligned} \tag{8}$$

其中 $\gamma, \beta \in \mathbb{R}^D$ 是可训练的逐特征缩放和平移参数，$\varepsilon > 0$ 用于稳定分母。求均值和方差的范围是同一位置的 D 个特征，因此对新位置做归一化不需要重新计算旧位置的统计量。并行实现包括行内归约，以及随后独立的逐元素变换。

逐位置前馈网络（position-wise feed-forward network，FFN）采用两次线性变换，中间加入非线性：

$$\mathrm{FFN}(H) = \phi(HW_1 + b_1)W_2 + b_2, \quad W_1 \in \mathbb{R}^{D\times D_{\mathrm{ff}}}, \quad W_2 \in \mathbb{R}^{D_{\mathrm{ff}}\times D}. \tag{9}$$

其中偏置分别按行广播。所有位置共享相同的 W_1, W_2，但各行的计算彼此独立。第一层把特征扩展到中间宽度，非线性改变特征组合的表达能力，第二层恢复到 D 维。若去掉非线性，两次线性变换可以合并为一次，无法表达这种非线性映射。跨位置的信息已经由注意力汇入当前行，前馈网络进一步变换该行的特征。

GPT-2 取 $D_{\mathrm{ff}} = 4D$，并采用高斯误差线性单元（Gaussian Error Linear Unit，GELU）。其

定义为 $\mathrm{GELU}(u) = u\Phi(u)$，其中 Φ 是标准正态分布的累积分布函数；常用的计算近似为

$$\mathrm{GELU}(u) \approx \frac{u}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(u + 0.044715u^3)\right)\right]. \tag{10}$$

复现一个既定模型时，需要匹配其使用的激活函数及近似表达。完成全部 L 层后，再对 $X^{(L)}$ 做最终层归一化，得到用于词表投影的 H。GPT-2 的词表投影与输入嵌入共享权重，即 $W_U = E^{\mathrm{T}}$，且 $b_U = 0$，从而将宽度 D 的表示映射到 V_{voc} 个分数。

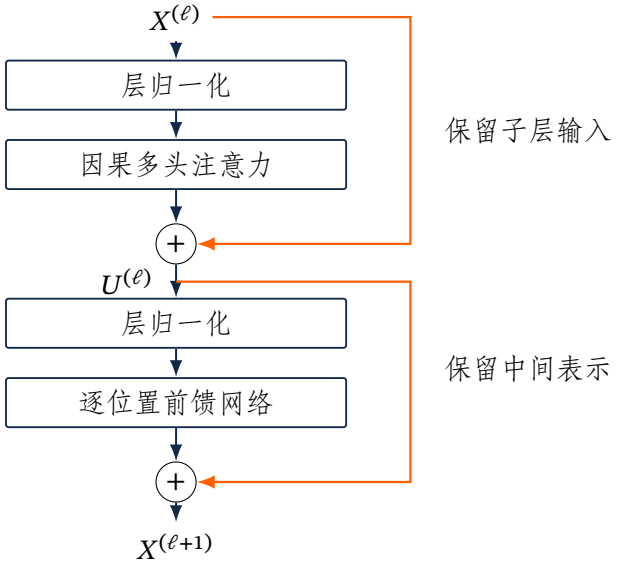


图 1 前置层归一化结构。每个子层先对输入归一化，再计算更新量，并通过残差路径加回未归一化的子层输入。所有残差相加均保持 $N \times D$ 的形状。

2.5 三个词元的注意力算例

考虑单头、三个位置、头维度 $d = 2$。为直接检查注意力内部的计算，给定已经投影得到的三个矩阵：

$$Q = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & \sqrt{2} \\ \sqrt{2} & \sqrt{2} \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}. \tag{11}$$

先计算每一行查询与各行键的内积，并除以 $\sqrt{2}$，得到

$$S = \frac{QK^{\mathrm{T}}}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \qquad S + M = \begin{bmatrix} 1 & -\infty & -\infty \\ 0 & 1 & -\infty \\ 1 & 1 & 2 \end{bmatrix}. \tag{12}$$

第一行只有位置 0 可见，因此权重恰为 $(1,0,0)$。第二行有效分数为 $(0,1)$，相应权重为 $(1/(1+e), e/(1+e), 0)$。第三行分数为 $(1,1,2)$，约去共同因子 e 后，权重为 $(1/(2+e), 1/(2+e), e/(2+e))$。因此

$$P \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.268941 & 0.731059 & 0 \\ 0.211942 & 0.211942 & 0.576117 \end{bmatrix}, \qquad O = PV \approx \begin{bmatrix} 1 & 0 \\ 0.268941 & 1.462117 \\ 1.940292 & 1 \end{bmatrix}. \tag{13}$$

例如第三个输出是 $0.211942(1,0) + 0.211942(0,2) + 0.576117(3,1)$。其第二个分量严格等于 $(2+e)/(2+e) = 1$，第一分量严格等于 $(1+3e)/(2+e)$。每行权重和为一，上三角为零，输出形状仍为 3×2。这些关系可以同时检查索引方向、掩码边界、归一化范围和值聚合是否正确。

3 注意力的 CUDA 实现

3.1 矩阵乘法流水线与存储布局

注意力中的投影、分数计算和值聚合都能归结为通用矩阵乘法（General Matrix Multiplication，GEMM）。可以调用 CUDA 基础线性代数库（cuBLAS），也可以用分块矩阵乘法核函数实现。逐行的指数归一化穿插在两次注意力矩阵乘法之间，需要额外处理归约与因果性。

表 2 单序列注意力的基线计算步骤。多头分数和输出按各头分别计算。

步骤	运算与形状	GPU 上的主要操作
三个投影	$(N \times D)(D \times 3D) \to N \times 3D$	可融合的矩阵乘法
分数计算	$(N \times d)(d \times N) \to N \times N$	矩阵乘法与 $1/\sqrt{d}$ 缩放
因果归一化	$S \to P$，保持 $N \times N$	行内最大值与求和归约
值聚合	$(N \times N)(N \times d) \to N \times d$	矩阵乘法
头合并与投影	$(N \times D)(D \times D) \to N \times D$	重排与矩阵乘法

缩放可以作为分数矩阵乘法的标量系数完成，从而避免再单独遍历 N^2 个元素。数学上的 K^{T} 也不必先物化成一个新矩阵；支持转置操作的矩阵乘法接口可以按照原缓冲区的布局读取。但行主序与列主序、转置标志以及前导维度必须相互一致，才能实现式 (5) 指定的乘法。

加入批量维后，输入常表示为 $[B, N, D]$。把融合投影的最后一维拆成 $[3, h, d]$，得到

$[B, N, 3, h, d]$。抽取查询部分后得到 $[B, N, h, d]$，再交换头轴与序列轴，形成 $[B, h, N, d]$。分数矩阵则是 $[B, h, N, N]$。维度重塑（reshape）可以只改变同一线性存储的解释；轴置换是否需要数据搬运，取决于后续核函数是否能直接处理相应的步长。布局选择会影响合并访存和矩阵乘法接口的使用方式。

以下核函数只处理一个头的一张分数矩阵。假定它按行主序连续存储，因此 S_{ij} 的地址偏移是 $iN + j$。输入 S 已经包含 $1/\sqrt{d}$ 缩放，但尚未归一化。输出 P 使用另一块同形状缓冲区，并将掩码位置明确置零，供后续稠密矩阵乘法使用。

3.2 数值稳定的因果归一化

直接计算 $\exp(S_{ij})$ 可能溢出。对同一行减去一个常数 c_i 不改变归一化结果，因为分子和分母中的公共因子 $\exp(c_i)$ 可以约去：

$$\frac{\exp(S_{ij})}{\sum_{k=0}^{i} \exp(S_{ik})} = \frac{\exp(S_{ij} - c_i)}{\sum_{k=0}^{i} \exp(S_{ik} - c_i)}. \tag{14}$$

取有效列中的最大值 $m_i = \max_{0 \le j \le i} S_{ij}$，所有参与计算的指数输入都不大于零，至少有一项指数等于一。定义行分母 s_i，得到

$$\begin{aligned} m_i &= \max_{0 \le j \le i} S_{ij}, \qquad s_i = \sum_{j=0}^{i} \exp(S_{ij} - m_i), \\ P_{ij} &= \begin{cases} \exp(S_{ij} - m_i)/s_i, & 0 \le j \le i, \\ 0, & i < j < N. \end{cases} \end{aligned} \tag{15}$$

在有效分数有限的条件下，$1 \le s_i \le i + 1$。这既避免指数上溢，也使分母具有明确的正下界。非常小的权重仍可能发生浮点下溢，但它们对加权和的影响相应较小。

最大值与分母的范围必须遵守同一掩码。例如，在行 $i = 1$，假设三个分数依次为 1000、1001 和 9000。只有前两列可见，正确计算取 $m_1 = 1001$，使用 $(e^{-1}, 1)$ 归一化，得到约 $(0.268941, 0.731059, 0)$。若错误地把未来列的 9000 纳入最大值，再只累加前两列，两项指数都可能下溢成零。若把未来列也纳入分母，则直接改变了因果分布。实现时遍历有效列，可以同时避免这两类问题。

这一步的并行结构不同于逐元素激活：同一行的每个输出都依赖该行共同的最大值和分母。因此必须先在行内归约，再把归约结果提供给所有负责该行的线程。

3.3 一行一个线程块的计算组织

令线程块包含 B_t 个线程，块编号 i = `blockIdx.x` 对应行号，线程编号 u = `threadIdx.x`。线程 u 按步长 B_t 访问 $j = u, u + B_t, u + 2B_t, \ldots$。前两次遍历只处理 $j \le i$；最后一次遍历覆盖 $j < N$，以写出完整的一行。

计算先由每个线程求自己负责部分的最大值，再在线程块内归约得到 m_i；随后，每个线程累加自己的 $\exp(S_{ij} - m_i)$，再次归约得到 s_i；最后，各线程独立写出概率。没有有效列的线程在最大值归约中贡献 $-\infty$，在求和归约中贡献零，仍然参与全部同步。这样，即使第一行只有一个有效元素，整个线程块的控制流程也保持一致。

例如取 $B_t = 256$，处理行 $i = 700$ 时，可见列为 $0, \ldots, 700$。线程 0 负责列 $0, 256, 512$；线程 189 负责 $189, 445$，下一项 701 已越过可见边界。因此，一行的长度可以大于线程块大小；增加长度表现为每个线程循环更多次。线程块数对应行数，也需要满足设备的网格维度限制。若把多个批次和多个头的行展平，则总行号 r 对应的因果位置应取 $i = r \bmod N$，不能直接把 r 当作掩码边界。

下面采用共享内存上的二叉树归约。设 B_t 为二的幂，线程先把局部结果写入共享数组；归约步长依次取 $B_t/2, B_t/4, \ldots, 1$。每一步中，前半部分线程组合两个位置的结果，再由块内屏障保证下一步读到已完成的写入。归约结束后，共享数组的第零项包含整行需要的统计量。

代码 1 线程块内的最大值或求和归约

```cpp
#include <cuda_runtime.h>
#include <math_constants.h>
#include <cmath>
#include <cstddef>
#include <limits>

// Contract: blockDim == (BT, 1, 1); every thread calls this helper.
// The final barrier lets the caller safely reuse the same scratch array.
template <int BT, bool ReduceMax>
__device__ __forceinline__ float block_reduce(float value, float* scratch
    ) {
    static_assert(BT > 0 && (BT & (BT - 1)) == 0,
                  "BT must be a positive power of two");
    const unsigned int tid = threadIdx.x;
    scratch[tid] = value;
    __syncthreads();

    for (unsigned int step = BT / 2; step > 0; step >>= 1) {
        if (tid < step) {
            const float other = scratch[tid + step];
            scratch[tid] = ReduceMax ? fmaxf(scratch[tid], other)
                                     : scratch[tid] + other;
        }
        __syncthreads();
    }
    const float result = scratch[0];
    __syncthreads();
    return result;
}
```

模板参数 ReduceMax 在编译时确定归约操作。每个线程都调用这个函数，包括局部结果为单位元的线程。循环结束时的屏障保证归约已经完成；随后先把 scratch[0] 读入每个线程自己的寄存器变量，再做一次屏障。这最后一次同步保证所有线程都已读取旧结果，后面的求和归约才可以安全地复用同一块共享内存。

核函数采用三次按行遍历。第二次遍历将指数临时写入输出缓冲区 P，第三次遍历再除以公共分母；这样每个有效位置只需计算一次指数。最后一次遍历使用相同的线程分工，因此每个线程归一化自己刚才写入的位置。P 的上三角部分尚未初始化，条件表达式会直接写零，不读取其中的旧内容。

代码 2 单头、单序列的因果归一化核函数

```
// One block per row. S already contains the 1/sqrt(d) scaling.
// S and P are disjoint row-major N-by-N device buffers; N >= 1.
// Only S[row, j] with j <= row must be finite; masked entries are not read.
template <int BT>
__global__ void causal_softmax_kernel(const float* __restrict__ S,
                                      float* __restrict__ P, int N) {
    __shared__ float scratch[BT];
    const std::size_t row = blockIdx.x;
    const std::size_t n = static_cast<std::size_t>(N);
    const std::size_t base = row * n;
    const std::size_t tid = threadIdx.x;

    // Inactive threads contribute the identity for a maximum reduction.
    float local_max = -CUDART_INF_F;
    for (std::size_t j = tid; j <= row; j += BT) {
        local_max = fmaxf(local_max, S[base + j]);
    }
    const float row_max = block_reduce<BT, true>(local_max, scratch);

    // P temporarily stores exponentials, owned by the same thread in both loops.
    // Every row includes its diagonal, so at least one exponential equals one.
    float local_sum = 0.0f;
    for (std::size_t j = tid; j <= row; j += BT) {
        const float value = expf(S[base + j] - row_max);
        P[base + j] = value;
        local_sum += value;
    }
    const float row_sum = block_reduce<BT, false>(local_sum, scratch);

    // The conditional avoids reading an uninitialized masked entry of P.
    for (std::size_t j = tid; j < n; j += BT) {
        P[base + j] = (j <= row) ? P[base + j] / row_sum : 0.0f;
    }
}
```

这里使用 `std::size_t` 计算行偏移，使乘积 iN 在较宽的无符号索引类型中形成。输入与输出缓冲区需要各自至少容纳 N^2 个单精度数，且二者的存储区间互不重叠。有效输入分数应为有限值；仅用于表示未来位置的上三角值不会被读取。

暂存指数和重新计算指数是两种保持同一数学结果的实现方式。暂存版本减少指数函数调用，同时增加一次中间结果写入；重新计算版本可以在最后一轮再次读取 S 并求指数。哪一种更快，取决于指数运算吞吐、显存带宽和缓存行为，需要按目标设备测量。上面的版本主要用于清楚展示归约与同步关系。

3.4 启动条件与实现检查

核函数通过一维网格启动 N 个线程块，每块固定为 256 个线程。启动之前应验证设备允许相应的网格大小、线程块大小和共享内存需求。共享数组只需 $256 \times 4 = 1024$ 字节，大小与 N 无关。较长上下文增加线程内循环次数和总行数，并不要求共享内存容纳整行。

代码 3 主机端的参数检查与异步启动

```
// The caller supplies disjoint buffers of at least N*N floats on this device.
// Returns argument/device/launch errors; does not wait for kernel completion.
cudaError_t launch_causal_softmax(const float* S, float* P, int N,
                                  cudaStream_t stream = nullptr) {
    constexpr int BT = 256;
    if (S == nullptr || P == nullptr || S == P || N <= 0)
        return cudaErrorInvalidValue;
    const std::size_t n = static_cast<std::size_t>(N);
    if (n > std::numeric_limits<std::size_t>::max() / sizeof(float) / n)
        return cudaErrorInvalidValue;

    int device = 0;
    cudaError_t status = cudaGetDevice(&device);
    if (status != cudaSuccess) return status;
    cudaDeviceProp prop{};
    status = cudaGetDeviceProperties(&prop, device);
    if (status != cudaSuccess) return status;
    if (N > prop.maxGridSize[0] || BT > prop.maxThreadsPerBlock ||
        BT > prop.maxThreadsDim[0] ||
        BT * sizeof(float) > prop.sharedMemPerBlock)
        return cudaErrorInvalidConfiguration;

    causal_softmax_kernel<BT><<<static_cast<unsigned int>(N), BT, 0, stream>>>(
        S, P, N);
    return cudaGetLastError();
}
```

这个辅助函数返回参数、设备查询或启动阶段发现的错误码，不等待核函数执行结束。同一流中的后续矩阵乘法按提交顺序读取 P；需要确认执行完成或测量耗时时，还应在合适的位置等待该流或记录设备事件。函数无法从裸指针确定缓冲区的实际容量，调用者仍需满足前述容量与不重叠条件。

正确性检查可以直接使用第 2.5 节的分数矩阵。预期输出为式 (13) 中的 P；随后计算 PV 应得到对应的 O。还应覆盖 $N = 1$、长度小于线程数、长度不等于线程数的整数倍以及长度大于线程数的情况。检查的对象包括有效行和接近一、上三角严格为零、输出有限，以及与高精度逐行参考计算的一致性。单精度归约改变加法顺序，通常会产生很小的舍入差异，应

根据数据类型比较数值误差。

性能上，一个线程块处理一行使行内通信局限于共享内存和块内屏障。相邻线程在同一次循环中访问相邻列，便于合并访存；早期行只有少量有效元素，部分线程参与同步但没有实际数据工作。更大的线程块可以减少长行的循环次数，也会增加线程和寄存器资源占用。线程数的选择需要同时考虑行长、活跃线程比例和设备并发能力。

4 两阶段推理与键值缓存

4.1 预填充与增量解码的时序

处理一个新的文本输入时，初始的多个词元都已知。预填充（prefill）把整个初始前缀转换为输入矩阵，并一次经过全部网络层。各位置虽然只能访问因果掩码允许的前缀，但可以一起进行投影、矩阵乘法和逐行归一化。最后一个位置的输出分布用于选择第一个新词元。

后续的增量解码（incremental decoding）每次只把刚生成的一个词元作为新的输入位置送入网络，同时复用历史位置已计算的状态。设初始前缀为 $t_0, \dots, t_{N-1}$。预填充完成后，每层都已经得到这 N 个位置的键和值；根据位置 $N-1$ 的最终输出，选择词元 t_N。此时词元 t_N 刚被选出，还没有经过网络，它的各层键和值尚未形成。

下一次前向传播把 t_N 放在绝对位置 N，逐层计算该新位置的表示和键值，并预测 t_{N+1}。所以“得到词元编号”与“形成该词元的各层缓存”发生在相邻的两个计算阶段。词元编号已经由词表选择直接得到，可以马上用于嵌入查表；生成回路不需要先把编号转为文本再分词。

表 3 初始前缀含 N 个词元时的前三次前向传播。缓存长度只统计已经输入网络的词元。

阶段	本次送入网络的词元	缓存行数	本次新词元
预填充	$t_0, \dots, t_{N-1}$	N	t_N
增量第 1 步	t_N，位置为 N	$N+1$	t_{N+1}
增量第 2 步	t_{N+1}，位置为 $N+1$	$N+2$	t_{N+2}

预填充对给定上下文进行表示计算，这里的阶段名称不表示模型必须执行文本摘要任务。聊天、续写、翻译等生成任务都可以采用同样的预填充与解码流程。

4.2 历史状态为什么可以复用

键值缓存（key-value cache，KV cache）保存已经计算出的历史键和值。它的正确性来自因果依赖，以及层内其他运算的逐位置性质。在模型参数固定、位置编号保持不变的确定性推理中，把未来词元追加到序列末尾，不会改变已有位置的表示。

这个结论可以按网络层数证明。首先，位置 i 的初始向量由 t_i 和位置 i 的嵌入相加得到。扩展序列不改变这两个量，所以旧位置的 $X_{i,:}^{(0)}$ 保持不变。假设第 ℓ 层所有旧位置的输入已经保持不变，考察该层位置 i 的计算。

层归一化只读取位置 i 的特征，因此它的结果不变，随后得到的 q_i, k_i, v_i 也不变。因果注意力仅使用位置 $0, \ldots, i$ 的键和值；新增的未来位置被屏蔽，因此第 i 行有效分数、归一化分母和加权输出都不变。输出投影、残差相加和前馈网络继续独立处理这一行，使第 $\ell + 1$ 层的旧位置输入仍然不变。由此逐层归纳，所有层中历史位置的键和值都可以保存后复用。

缓存必须按层保存，因为第 ℓ 层的键和值由该层自己的输入与投影权重产生。只保存初始嵌入无法代替中间层的键值，只保存最后一层也不能支持前面各层的注意力。标准多头结构中，每层保存 $K^{(\ell)}, V^{(\ell)} \in \mathbb{R}^{B \times h \times N \times d}$；不同层的数值不同，尽管形状相同。

未来位置使用的是新的查询，它需要与历史键进行匹配，再对历史值做加权和。旧查询只用于生成旧位置的输出，因此标准增量解码不需要缓存历史 Q。同样，也不需要保存全部历史注意力概率矩阵。缓存 K/V 保留了后续新查询所需的历史信息，同时避免存储会随位置数平方增长的概率状态。

如果修改已有前缀、改变已有位置编号或替换模型参数，上述不变性条件就发生变化，受影响的缓存需要重新建立。这一条件也说明：缓存的有效性取决于生成时的实际计算依赖。

4.3 一个新增位置在各层中的计算

设某层、某头已经缓存 N 个位置。对于新位置 N，令 $x_N \in \mathbb{R}^{1 \times D}$ 表示送入该头投影的归一化输入。只对这一行做三个投影，并将新键和值追加到对应的历史缓存：

$$
\begin{aligned}
&q_N = x_N W_Q, \qquad k_N = x_N W_K, \qquad v_N = x_N W_V, \\
&\widehat{K} = \begin{bmatrix} K_{0:N,:} \\ k_N \end{bmatrix}, \qquad \widehat{V} = \begin{bmatrix} V_{0:N,:} \\ v_N \end{bmatrix}, \\
&o_N = \mathrm{softmax}\left(\frac{q_N \widehat{K}^{\mathrm{T}}}{\sqrt{d}}\right)\widehat{V}.
\end{aligned}
\tag{16}
$$

切片 $0:N$ 表示包含 0、不包含 N，即原来的 N 行。新查询的形状为 $1\times d$，扩展后的缓存为 $(N+1)\times d$，分数和权重为 $1\times(N+1)$，输出为 $1\times d$。当前新位置自己的键和值已经包含在缓存中，因此对角线也被正确保留。

当一次只新增一个位置，且缓存仅含当前与过去位置时，缓存中的全部列都满足因果条件，不再需要显式构造一个三角掩码。若存在填充位置或其他可见性限制，仍需按照这些限制屏蔽相应列。若一次追加多个位置，则每个新增位置的可见列数不同，仍需要带历史长度偏移的因果掩码。

第 3 节的核函数使用方阵中的局部行号作为因果边界。单词元解码的分数矩阵只有一行，但这一行对应的绝对位置是 N，其可见范围是 $0,\dots,N$。因此，把这个长向量作为局部第零行直接套用方阵核函数，只会错误地保留第一列。解码核函数需要使用实际可见长度，或者显式加入历史长度偏移。

所有头只计算新行之后，拼接它们的输出，再经过 W_O、残差、归一化和前馈网络，得到下一层的新行。下一层使用自己保存的 K/V 重复相同过程。完成全部层后，最后归一化与词表投影只需处理这个新位置，从而得到再下一个词元的分布。

第 2.5 节可以直接展示这种复用。先处理前两个位置，缓存键 $(1,0),(0,1)$ 和值 $(1,0),(0,2)$。追加第三个位置时，只需新的 $q_2=(\sqrt{2},\sqrt{2})$、$k_2=(1,1)$、$v_2=(3,1)$。新查询与三行键的缩放分数为 $(1,1,2)$，因此新输出为 $(1.940292,1)$，与一次处理全部三行的结果相同。前两个输出仍然是 $(1,0)$ 和 $(0.268941,1.462117)$，无需重新计算。

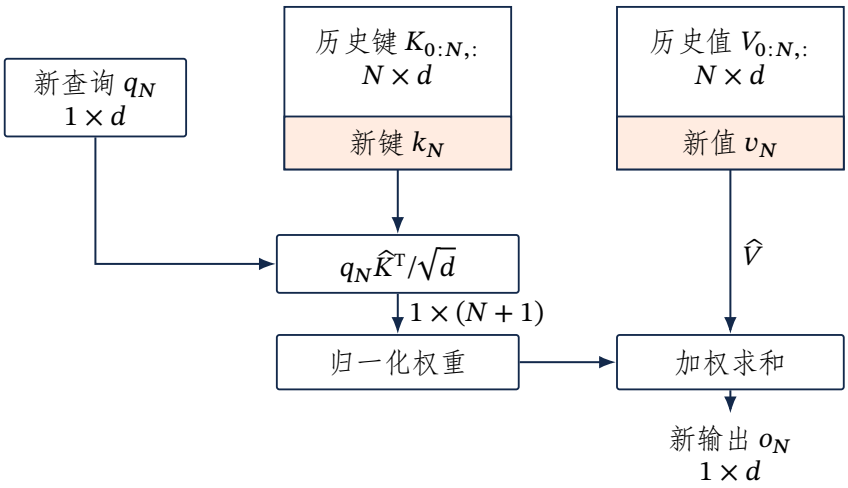


图 2 单层单头的增量注意力。历史键值保留，新键值追加为最后一行；只用新查询计算一个长度为 $N+1$ 的权重向量，再形成新输出。

4.4 节省的计算与增加的状态容量

比较两种实现处理到相同总长度 N 时，产生下一词元分布所做的工作。直接重算方式把全部 N 个位置重新输入网络；缓存方式只处理新增的一行，并访问已有键值。单头的主要差别如表 4 所示。

表 4 单头注意力在总长度 N 时的计算与持久状态。省略不影响阶数的常数。

对象	重算完整前缀	增量解码
查询、键和值投影	$\Theta(NDd)$	$\Theta(Dd)$
分数与值聚合	$\Theta(N^2d)$	$\Theta(Nd)$
注意力权重的计算行数	全部 N 行	新增的 1 行
持久注意力状态	不保存历史状态	$2Nd$ 个键值元素

缓存把重复计算历史位置的工作去掉了，但新查询仍要访问全部可见键值。因此，每步的注意力计算和缓存读取量仍随 N 线性增长。所有头的 $hd = D$，所以每层的新键值投影为 $\Theta(D^2)$ 量级，注意力匹配与聚合为 $\Theta(ND)$ 量级。输出投影与前馈网络也只计算新行，不再对历史行重复执行。

缓存容量由两个矩阵、批量、层数、上下文长度和特征宽度共同决定。在标准多头注意力中，若每个缓存元素占 b_{elem} 字节，则

$$n_{\text{KV}} = 2BLhNd = 2BLND, \qquad M_{\text{KV}} = 2BLND\, b_{\text{elem}} \text{ 字节}. \tag{17}$$

这个公式假定每个查询头都有对应的独立键和值头。它只统计已经使用的缓存元素；按最大长度预分配的实现，应把 N 替换为分配容量对应的长度。模型权重、其他激活和临时工作区还会额外占用显存。

例如，取 $B = 1$、$L = 32$、$D = 4096$、$h = 32$、$d = 128$、$N = 4096$，并用 16 位数保存缓存，即每个元素占 2 字节，则

$$M_{\text{KV}} = 2 \times 1 \times 32 \times 4096 \times 4096 \times 2 = 2\,147\,483\,648 \text{ 字节} = 2 \text{ GiB}. \tag{18}$$

这里 $1 \text{ GiB} = 2^{30}$ 字节。这是一组说明容量关系的标准多头配置。保持其他参数不变时，上下文长度或并行序列数加倍，已使用的缓存容量也加倍；网络层数增加同样线性增加缓存。

工程实现可以为每层预分配足够的键值空间，并用当前有效长度指示已写入部分。追加时只写新行，避免每步重新分配并复制全部历史缓存。若底层采用分块或分页存储，其逻辑内容仍然是式 (16) 中按位置排列的历史键值。

4.5 变长序列与键值头共享

一个批量中的不同序列通常具有不同的有效长度。设第 b 个序列的长度为 N_b，则标准多头结构实际保存的键值数据为

$$M_{\mathrm{KV}} = 2Lhd\, b_{\mathrm{elem}} \sum_{b=1}^{B} N_b. \tag{19}$$

如果为每个序列按统一上限 N_{cap} 预分配，则分配量为 $2BLhN_{\mathrm{cap}}d\, b_{\mathrm{elem}}$，可能大于有效数据量。按块分配可以减少未使用空间；块表、对齐以及最后一块中的空余位置仍会产生额外开销。不同请求的历史词元和中间表示通常不同，因此不能把一个请求的缓存直接用作另一个请求的缓存。

标准多头注意力为每个查询头配置独立的键值头。也可以让多个查询头共用一组键和值。设查询头数为 h_q、键值头数为 h_{kv}，每个头的维度为 d，模型宽度仍为 $D = h_q d$。等长批量的缓存容量统一写为

$$M_{\mathrm{KV}} = 2BLNh_{\mathrm{kv}}d\, b_{\mathrm{elem}}. \tag{20}$$

多头注意力（MHA）取 $h_{\mathrm{kv}} = h_q$；分组查询注意力（grouped-query attention，GQA）取 $1 < h_{\mathrm{kv}} < h_q$，通常让 h_{kv} 整除 h_q，每组查询头共享一个键值头；多查询注意力（multi-query attention，MQA）取 $h_{\mathrm{kv}} = 1$，所有查询头共享键和值。各查询头仍保留自己的查询，计算各自的匹配分数和输出。

在式 (18) 的配置中，32 个查询头若对应 8 个键值头，缓存由 2 GiB 降至 0.5 GiB。同一组内的查询还可以复用读入的键值数据，实际复用效果取决于核函数组织。这些结构使用不同形状的投影参数，必须与模型及预训练权重匹配；既定 GPT-2 权重的推理仍采用其原有多头结构。

5 计算规模与端到端性能

5.1 矩阵形状如何决定运算量

浮点运算次数（floating-point operations，FLOP）的计数需要先固定约定。把一次实数乘法和一次实数加法各计为一次运算，则一次融合乘加（fused multiply-add，FMA）计为 2 FLOP。对于 $(m \times k)(k \times n)$ 的稠密矩阵乘法，主导运算量按 $2mkn$ 计算。下面只统计主要矩阵乘法，暂不计偏置、归一化、激活和采样中的低阶或特殊函数开销。

在标准多头结构 $hd = D$ 下，所有头的三个输入投影合计为 $6ND^2$ FLOP，输出投影为 $2ND^2$ FLOP。单头 QK^{T} 需要约 $2N^2d$ FLOP，所有头合计 $2N^2D$；PV 同样需要 $2N^2D$。前馈网络的两次乘法合计 $4NDD_{\mathrm{ff}}$。由此得到表 5。

表 5 单序列、单层的主要矩阵乘法开销。预填充列采用完整稠密矩阵乘法的计数，解码列的可见长度为 N。

计算部分	长度 N 的预填充	一个位置的增量解码
全部查询、键和值投影	$6ND^2$	$6D^2$
分数矩阵 QK^{T}	$2N^2D$	$2ND$
值聚合 PV	$2N^2D$	$2ND$
多头输出投影	$2ND^2$	$2D^2$
前馈两次投影	$4NDD_{\mathrm{ff}}$	$4DD_{\mathrm{ff}}$

因此，一层预填充的主要算量为 $8ND^2 + 4N^2D + 4NDD_{\mathrm{ff}}$。取 $D_{\mathrm{ff}} = 4D$，可写成

$$F_{\mathrm{prefill}} \approx 24ND^2 + 4N^2D, \qquad F_{\mathrm{decode}} \approx 24D^2 + 4ND. \tag{21}$$

对包含 L 层的网络，层内部分再乘 L。最终词表投影需要另外计入：若计算全部位置的分布，需要约 $2NDV_{\mathrm{voc}}$ FLOP；纯生成的预填充只需最后一个位置的词表分布，可以把这部分降为约 $2DV_{\mathrm{voc}}$，而网络内部仍需计算各层的前缀键值。

式 (21) 的二次项来自位置两两匹配与聚合。若实现真正跳过上三角，则 QK^{T} 与 PV 的有效计算合计约为 $2N(N+1)D$，常数约减半；直接调用完整的稠密矩阵乘法时，上三角即使在归一化后为零，也不会自动省掉乘法。两种实现仍具有 $\Theta(N^2D)$ 的预填充算术阶数。

模型宽度也决定主导开销。举例而言，若 $N = D$ 且 $D_{\mathrm{ff}} = 4D$，完整稠密基线中位置匹配与聚合占主要矩阵乘法算量的 $4/(24+4) = 1/7$。长度增加时这一占比上升。因此，判断耗时需要同时考虑 N、D、前馈宽度和实际核函数效率，不能只根据某一步含有 N^2 就决定优化顺序。

5.2 大规模投影的参数与容量算例

以 GPT-3 的大规模配置中的投影尺寸为例，取 $D = 12\,288$、$h = 96$、$d = 128$、$L = 96$。这里只统计每层三个注意力输入投影及一个输出投影，忽略偏置。单头的一个输入投影有 $Dd = 12\,288 \times 128 = 1\,572\,864$ 个参数；全部头的三个投影合计 $3hDd = 3D^2$。输出矩阵有 D^2 个参数，因而 L 层共有

$$n_{\text{proj}} = L(3D^2 + D^2) = 4LD^2 = 57\,982\,058\,496. \tag{22}$$

它约为 58.0×10^9 个注意力投影参数。每个新位置都要使用这些投影，按每次乘加 2 FLOP 计，其主要算量为 $2n_{\text{proj}} \approx 115.964 \times 10^9$ FLOP。

表 6 给定宽度和层数下的投影规模。参数数为精确整数，运算量按乘加计数后取近似显示。

对象	参数数	16 位容量	单位置算量
单头的一种输入投影	1 572 864	3 MiB	3.146 MFLOP
单层全部输入投影	452 984 832	864 MiB	0.906 GFLOP
单层输出投影	150 994 944	288 MiB	0.302 GFLOP
96 层四类投影	57 982 058 496	108 GiB	115.964 GFLOP

容量按每个参数 2 字节计算，其中 $1\ \text{MiB} = 2^{20}$ 字节、$1\ \text{GiB} = 2^{30}$ 字节；运算量前缀采用十进制，即 $1\ \text{MFLOP} = 10^6$ FLOP、$1\ \text{GFLOP} = 10^9$ FLOP。显存容量的二进制单位与运算计数的十进制前缀需要分别处理。

这些结果仅来自四类投影矩阵。上下文相关的匹配与聚合、前馈网络、词表投影以及各类中间状态都还未计入，因此 108 GiB 不是完整模型的总显存需求，115.964 GFLOP 也不是生成一个词元的全部计算量。这个算例的用途是把矩阵尺寸、参数存储和单位置算量对应起来。

5.3 预填充与解码的资源需求

预填充中的投影具有许多输入行，同一权重块可以服务多个位置，适合用分块矩阵乘法增加数据复用。单序列增量解码只有一行输入，投影退化为向量与矩阵乘法，单次读入的权重参与的计算更少。批量处理多个序列可以增加同时参与计算的行数，但同时也会增加各序列的缓存容量和键值读取量。

从算术强度（arithmetic intensity）可以看出这个差别。假设一个投影权重矩阵含 Dd 个元素，本次需要把它们全部从显存读取，每元素占 b_{elem} 字节。只处理一个新位置时，主要算量为 $2Dd$，仅对权重读取量计算的算术强度约为 $2/b_{\text{elem}}$ FLOP/字节。若以 16 位保存权重，

这个比值约为 1 FLOP/字节。多个输入行共同使用权重块时，可以把这一次读取摊到更多乘加操作上。

若同时统计输入读取和输出写入，可进一步量化批量带来的复用。设一个线性层的权重形状为 $m \times n$，处理 B 行输入。假定输入、权重各读取一次，输出写入一次，每个元素占 b_{elem} 字节，忽略偏置和其他操作，则

$$AI_{\text{linear}} \approx \frac{2Bmn}{b_{\text{elem}}(mn + Bm + Bn)}. \tag{23}$$

取 $m = 1024$、$n = 4096$、$b_{\text{elem}} = 2$，$B = 1$ 时约为 0.999 FLOP/字节；$B = 512$ 时约为 315.08 FLOP/字节。输入行数增多使权重读取被更多计算复用，而输入和输出的数据量也随之增长。因此，这个完整传输估计与前面只统计权重读取的估计具有不同分母。真实实现若重复读取数据，算术强度还会降低。

不同请求可以使用同一组投影权重，但通常必须读取各自的键值缓存。增加请求批量不会自动减少每个查询所需的历史键值读取，也不会自动提高这一部分按缓存读取计算的算术强度；它仍可能通过增加并行工作改善设备利用率。批量扩大时，还需同时检查缓存容量、序列长度差异及调度开销。

注意力读取缓存也具有相似的限制。单头一个新查询的匹配与聚合共约 $4Nd$ FLOP，需要读取约 $2Nd$ 个历史键值元素；只考虑这些读取时，算术强度同样约为 $2/b_{\text{elem}}$。这解释了为什么缓存虽然减少计算，较长上下文的逐词元解码仍可能受显存带宽限制。实际缓存命中、并行批量、数据类型和核函数融合都会影响最终表现。

设某段运算需要 F FLOP、传输 M 字节，设备在相应数据类型上的峰值算术吞吐为 R_{comp}、显存带宽为 R_{mem}，则理想时间至少满足

$$T \geq \max\left(\frac{F}{R_{\text{comp}}}, \frac{M}{R_{\text{mem}}}\right). \tag{24}$$

真实时间还包含无法完全重叠的访存、同步、启动和布局转换等开销。因此，算量减少多少与最终时间减少多少之间，还隔着硬件资源利用和实现方式。

5.4 分块融合如何减少中间矩阵读写

基线预填充通常显式生成分数矩阵 S 和权重矩阵 P。每个头每张矩阵包含 N^2 个元素，矩阵乘法、归一化和值聚合在相继的核函数中对它们写入和读取。例如 $B = 1$、$h = 32$、$N = 4096$ 时，一张所有头合计的矩阵有 $32 \times 4096^2 = 536\,870\,912$ 个元素；若以 32 位保存，仅这一张矩阵就占 2 GiB，同时保留 S 和 P 则需要 4 GiB。这部分是单层的临时状态，与跨层持久保存的 KV 缓存承担不同的作用。

闪电注意力（FlashAttention）通过分块与核函数融合，在片上保存局部查询、键值和中间统计量，使完整的 $N \times N$ 分数与概率矩阵不必写回显存。对一组查询，逐块读取键和值，计算局部分数、更新归一化统计量，并累积输出。它改善的是数据搬运和中间存储；完整稠密注意力需要处理的位置关系仍然是二次规模。

分块计算要保持相同的归一化结果，需要在读取后续块时正确调整已经累积的量。对某一个固定查询，设已经处理的位置集合为 A，分数为 s_j。保存最大值 $m = \max_{j \in A} s_j$、指数和 $\lambda = \sum_{j \in A} e^{s_j - m}$，以及未归一化输出向量 $u = \sum_{j \in A} e^{s_j - m} v_j$。当前输出就是 u/λ。

假设随后读入另一个非空可见块 J，其最大分数为 m_J。新的公共最大值取 $m' = \max(m, m_J)$。由于 $e^{s_j - m'} = e^{m - m'} e^{s_j - m}$，旧的累计量只需乘同一个校正因子，就能与新块合并：

$$
\begin{aligned}
m' &= \max(m, m_J), \qquad \alpha = \exp(m - m'), \\
\lambda' &= \alpha \lambda + \sum_{j \in J} \exp(s_j - m'), \\
u' &= \alpha u + \sum_{j \in J} \exp(s_j - m') v_j .
\end{aligned}
\tag{25}
$$

先用第一个非空可见块初始化这些量，再反复使用此递推；因果掩码下没有可见元素的块直接跳过。读完全部可见位置后，输出 u/λ，即与整行归一化后加权求和相同的数学结果。有限精度下，运算顺序的改变可能带来舍入差异。这个递推说明了为什么只保留行统计量与一个长度为 d 的累加向量，就能避免存储整行的全部概率。

KV 缓存复用不同生成步骤之间已经计算的历史键值，分块融合则减少同一次注意力计算内部的显存往返。这两个方向可以共同使用：预填充高效地建立各层缓存，后续增量解码继续访问这些缓存，并使用适合单查询或小批量形状的核函数。

5.5 从核函数性能到完整生成时间

完整推理包括初始分词和嵌入、预填充、重复的增量解码、词表选择以及必要的数据传输。首个词元的延迟主要涉及初始输入处理和预填充；后续逐词元延迟取决于增量路径。测量时应明确输入长度、输出长度、批量和数据类型，并将这两个阶段分别观察，再统计完整生成任务的时间。

局部加速对整体性能的影响由它在总时间中的占比决定。若某部分原先占总时间的比例为 f，优化后加速 r 倍，其余部分耗时保持不变，则整体加速比为

$$R_{\text{total}} = \frac{1}{(1-f) + f/r}. \tag{26}$$

例如某部分占原时间的 60%，即使该部分加速 4 倍，整体加速也只有 $1/(0.4 + 0.6/4) \approx 1.82$ 倍。这个假设算例说明，优化时应结合实际耗时占比评估收益；核函数之间多出的缓冲区转换或同步也可能抵消局部收益。

给定预训练权重的 GPT-2 推理项目可以围绕这条计算链展开。第一阶段把已有 CPU 参考实现对应的基本运算移到 GPU，并建立数值一致性；第二阶段分析单个核函数和完整推理流程的耗时，提出并实现相应优化；第三阶段继续优化并分析完整生成任务的性能。预训练参数保持固定，工作重点是正确实现和加速前向推理。

从计算依赖来看，投影和前馈主要改变特征表示，因果注意力把历史信息汇入当前位置，逐行归一化把分数变为稳定权重，键值缓存让已完成的位置计算得到复用。对应到实现，矩阵乘法的数据复用、归约的同步组织、缓存的按层维护以及中间结果的读写，共同决定完整生成过程的性能。